Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа №1
«Интерфейсы ввода-вывода общего назначения (GPIO)»

по дисциплине «Проектирование вычислительных систем»

Вариант 1. Пешеходный светофор

Студенты:
Анкудинов Кирилл Константинович
Кортыш Константин Олегович
Группа:
P3318
Преподаватель:
Пинкевич Василий Юрьевич

Санкт-Петербург, 2026

Оглавление

1. Цель работы 3
2. Постановка задачи 3
3. Архитектура решения 4
4. Детали реализации 5
   4.1. Главный цикл 5
   4.2. Драйвер GPIO 5
   4.3. Модуль светодиодов 6
   4.4. Модуль кнопки 6
   4.5. Конечный автомат светофора 6
5. Выводы 7

1. Цель работы

Реализовать на стенде SDK-1.1M с микроконтроллером STM32F427VI алгоритм пешеходного светофора по варианту 1. Для этого нужно настроить выводы GPIO кнопки и светодиодов, отделить прикладную логику от работы с пинами и организовать смену фаз без блокирующего ожидания: пока горит текущий сигнал, программа должна по-прежнему видеть нажатие кнопки.

Вторая часть работы — заменить обращения к GPIO через HAL собственным драйвером и работать с регистрами порта напрямую, сохранив прежнее поведение светофора.

2. Постановка задачи

Светофор циклически переключает сигналы в порядке: красный, зелёный, мигающий зелёный, жёлтый. Длительность красного по умолчанию составляет 12 секунд. Зелёный горит в четыре раза меньше, то есть 3 секунды. Длительности мигающего зелёного и жёлтого в задании не фиксированы; в программе приняты 2 секунды и 1,5 секунды. Полупериод мигания зелёного равен 250 мс, полный период вспышки — 0,5 с.

Кнопка на боковой панели запрашивает более ранний переход к зелёному, не ломая порядок фаз:

- нажатие во время красного сокращает остаток этой фазы до 3 секунд, если до конца осталось больше 3 секунд; иначе длительность не меняется;
- нажатие во время мигающего зелёного или жёлтого запоминается и сокращает ближайший красный до 3 секунд; следующий красный снова длится 12 секунд, если кнопку не нажали ещё раз;
- нажатие во время сплошного зелёного игнорируется.

Драйверы светодиодов и кнопки не содержат логики светофора и не блокируют выполнение: в них нет ожидания нажатия и вызовов `HAL_Delay()`. Опрос кнопки включает программную защиту от дребезга. Прерывания по кнопке и дополнительные таймеры, кроме SysTick, не используются.

Режимы выводов, чтение и запись уровней выполняет собственный драйвер `gpio_drv`. Функции HAL и LL для настройки и использования GPIO в программе не вызываются. Допускается только описание регистров `GPIO_TypeDef` и константы вида `GPIO_PIN_13` из CMSIS.

Таблица 1. Фазы светофора

Фаза	**Длительность**	**Что горит**
Красный	12 с или 3 с	красный
Зелёный	3 с	зелёный
Зелёный мигающий	2 с	зелёный, период 0,5 с
Жёлтый	1,5 с	жёлтый

3. Архитектура решения

Программа выполняется без операционной системы реального времени. После инициализации периферии `main` входит в бесконечный цикл и на каждом проходе вызывает `traffic_update()`. Время берётся из SysTick: `HAL_GetTick()` возвращает число миллисекунд с момента старта. Функция не ждёт окончания фазы, а сравнивает текущее время с моментом входа в неё.

Обращение к выводам сосредоточено в драйвере GPIO и двух модулях над ним. Модуль светофора знает только логические имена индикаторов и событие нажатия кнопки. `led_init()` переводит PD13–PD15 в выход push-pull, `button_init()` делает PC15 входом без внутренней подтяжки. Подтяжку кнопки к 3,3 В даёт резистор на плате, нажатие замыкает PC15 на землю. В `lab1_var1.ioc` эти пины не назначены, чтобы генератор CubeMX не вернул инициализацию через HAL.

Таблица 2. Модули прикладной программы

Модуль	**Ответственность**
`gpio_drv.c` / `gpio_drv.h`	Тактирование порта, режим выводов, чтение текущих настроек, чтение и запись одного сигнала и сразу нескольких пинов одного порта через регистры MODER, IDR и BSRR.
`led.c` / `led.h`	Включение и выключение зелёного, красного и жёлтого индикаторов. Соответствие имён и выводов PD13, PD14, PD15 хранится только здесь.
`button.c` / `button.h`	Неблокирующий опрос PC15 и одно событие нажатия после антидребезга 20 мс.
`traffic.c` / `traffic.h`	Конечный автомат фаз, реакция на кнопку и выбор того, какие индикаторы горят.

Состав модулей приведён в Таблица 2. Связь между ними показана на Таблица 3: светофор опрашивает кнопку и отдаёт команды драйверу индикаторов, а тот пишет уровни на порт D через `gpio_drv`.

Таблица 3. Один проход главного цикла

`traffic_update()`
опрос `button_poll()`
если фаза истекла — переход к следующей
обновление светодиодов через `led_set()`

Тактирование ядра настроено от внешнего кварца HSE 25 МГц. PLL умножает её до SYSCLK 180 МГц (коэффициенты PLLM = 25, PLLN = 360, PLLP = 2), для этой частоты включён режим over-drive. Миллисекундный тик SysTick считается уже от полученной частоты, поэтому длительности фаз в Таблица 1 не зависят от того, сколько команд выполняется в цикле.

4. Детали реализации

4.1. Главный цикл

После `led_init()`, `button_init()` и `traffic_init()` выполняется бесконечный цикл. На каждом проходе вызывается `traffic_update()` без задержек.

```
led_init();
button_init();
traffic_init();

while (1)
{
  traffic_update();
}
```

4.2. Драйвер GPIO

Драйвер работает с регистрами порта напрямую. `gpio_drv_init` настраивает режим, тип выхода, скорость и подтяжку; для выхода уровень пишется в `BSRR` до смены `MODER`, чтобы не было выброса. `gpio_drv_read` и `gpio_drv_write` читают `IDR` и пишут `BSRR` для одного пина и для маски. `gpio_drv_get` возвращает текущие настройки.

```
static void write_field(volatile uint32_t *reg, uint16_t pins, uint32_t value)
{
  uint32_t mask = field_mask(pins);
  *reg = (*reg & ~mask) | field_value(pins, value);
}

static void write_level(GPIO_TypeDef *port, uint16_t pins, bool on)
```

```
{
  if (on)
    port->BSRR = pins;
  else
    port->BSRR = (uint32_t)pins << 16;
}
```

4.3. Модуль светодиодов

`led_init` настраивает PD13–PD15 через `gpio_drv`. Соответствие индикаторов выводам задано таблицей `leds`. Красный и жёлтый включены встречно между PD14 и PD15, поэтому при включении одного цвета второй гасится.

```
static const struct {
  GPIO_TypeDef *port;
  uint16_t pin;
} leds[LED_COUNT] = {
  [LED_GREEN]  = { GPIOD, GPIO_PIN_13 },
  [LED_RED]    = { GPIOD, GPIO_PIN_15 },
  [LED_YELLOW] = { GPIOD, GPIO_PIN_14 },
};

void led_set(led_id_t led, bool on)
{
  gpio_drv_write(leds[led].port, leds[led].pin, on);
}
```

4.4. Модуль кнопки

`button_init` настраивает PC15 как вход без подтяжки: её даёт резистор на плате, нажатие замыкает линию на землю. `button_poll` опрашивает пин через `gpio_drv_read` и подтверждает нажатие после антидребезга 20 мс.

```
static bool read_raw(void)
{
  return !gpio_drv_read(BUTTON_PORT, BUTTON_PIN);
}

if (raw != last_raw) {
  last_raw = raw;
  raw_changed_at = now;
  return BUTTON_EVENT_NONE;
}
if (raw == stable_pressed || (now - raw_changed_at) < DEBOUNCE_MS)
  return BUTTON_EVENT_NONE;
stable_pressed = raw;
```

4.5. Конечный автомат светофора

Фазы заданы перечислением `phase_t` и массивом длительностей `phase_ms`. Переход выполняет `enter_phase`. Нажатие во время красного сокращает остаток до 3 с, во время мигающего зелёного или жёлтого ставит `green_requested`, во время зелёного игнорируется.

```
static void enter_phase(phase_t next, uint32_t now)
{
  phase = next;
  phase_start = now;
  phase_len = phase_ms[next];
  if (next == PHASE_RED && green_requested) {
    phase_len = RED_SHORT_MS;
    green_requested = false;
  }
}

void traffic_update(void)
{
  uint32_t now = HAL_GetTick();
  if (button_poll() == BUTTON_EVENT_PRESS)
    on_button_press(now);
  if ((now - phase_start) >= phase_len)
    enter_phase((phase_t)((phase + 1) % PHASE_COUNT), now);
  show(now - phase_start);
}
```

5. Выводы

На стенде SDK-1.1M реализован пешеходный светофор варианта 1. Фазы идут в заданном порядке: красный 12 с, зелёный 3 с, мигающий зелёный 2 с с периодом 0,5 с, жёлтый 1,5 с. Кнопка на PC15 сокращает текущий красный, если до его конца больше 3 с, либо запоминает запрос на короткий красный при нажатии во время мигания или жёлтого.

Номера выводов и чтение кнопки скрыты в драйверах `led` и `button`. Прикладной автомат в `traffic` от них не зависит и не блокирует цикл: ожидание фаз построено на сравнении `HAL_GetTick()` с моментом входа в фазу, а дребезг отфильтровывается по устойчивости уровня в течение 20 мс. Режимы GPIO задаёт собственный драйвер записью в `MODER`, `IDR` и `BSRR`. Тактовая частота 180 МГц от кварца 25 МГц по-прежнему настраивается через HAL и STM32CubeMX: это дерево тактирования, а не порты ввода-вывода.